

Mathematical Prerequisites

What this course assumes, and where to get it

NLP and LLMs · Week 1 handout · Hung-Shin Lee · 2026-09-22 · read before Week 3

This course has no formal prerequisites, and nothing here is examined. But from Week 1 onward the derivations use the objects below *without stopping to define them*. This sheet lists each object, the one fact about it the course actually uses, and where that fact shows up. It is a map, not a textbook: read **UDL** for the material.

UDL = Prince, *Understanding Deep Learning*, free PDF with notebooks and an answer booklet at udlbook.github.io/udlbook — pin **v5.0.3 (2026-02-09)**, since chapter numbers can move between versions. **ZZM ch 2** (*Basics of Neural Network*, open access) covers the same ground more briefly.

Notation used throughout the course. Bold lowercase $\mathbf{x}$ is a vector, bold uppercase $\mathbf{W}$ a matrix, $\mathcal{L}$ a loss. $\log$ is natural unless written $\log_2$. The letter σ does double duty, as everywhere in this field: $\sigma(x) = 1/(1 + e^{-x})$ is the sigmoid, and a subscripted $\sigma_{\max}(\mathbf{W})$ or σ_i is a singular value. Context always disambiguates.

1. Linear algebra — *UDL Appendix B*

Dot product, norm, cosine. $\mathbf{u}^\top \mathbf{v} = \sum_i u_i v_i = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$. A dot product is a *score of agreement* between two vectors, coordinate by coordinate. Cosine similarity is that score with the lengths divided out. *Used:* the similarity between two word embeddings (W1, demo 3), the attention score $\mathbf{q}^\top \mathbf{k}$ (W3), the retrieval score in RAG (W12). The fact to hold on to: the coordinates were chosen by a training objective, so cosine measures *whatever that objective rewarded* — not meaning. That is why antonyms come out similar in W1.

Matrix as a map, and the cost of applying it. $\mathbf{W} \in \mathbb{R}^{m \times n}$ sends $\mathbb{R}^n \rightarrow \mathbb{R}^m$; computing $\mathbf{W}\mathbf{x}$ costs one multiply and one add per entry, $2mn$ FLOPs. That single count, applied to every weight in a model, gives the $2N$ FLOPs per token for inference and $6ND$ for training (W1, the compute ledger; W4; W7; W11).

Projection. The component of $\mathbf{u}$ along a unit vector $\mathbf{d}$ is $(\mathbf{u}^\top \mathbf{d}) \mathbf{d}$. The residual stream of W4 is a vector that every layer writes into and reads from by projection, and reading a direction out of it is how probes and W14's interpretability tools work.

Singular values and the SVD. Every matrix factors as $\mathbf{W} = \mathbf{U}\Sigma\mathbf{V}^\top$ with Σ diagonal and non-negative. Two facts carry the whole course:

- $\sigma_{\max}(\mathbf{W})$ is the *largest stretch* $\mathbf{W}$ applies to any unit vector: $\|\mathbf{W}\mathbf{x}\| \leq \sigma_{\max} \|\mathbf{x}\|$. Hence $\|\mathbf{W}_T \cdots \mathbf{W}_1\| \leq \prod_t \sigma_{\max}(\mathbf{W}_t)$ for a product of *different* matrices. This is the bound behind vanishing and exploding gradients (W1, the BPTT slide) — and it is a bound on singular values, not eigenvalues. Eigenvalues only govern powers of *one* square matrix; the matrix $\begin{pmatrix} 0 & 10 \\ 0 & 0 \end{pmatrix}$ has both eigenvalues zero and $\sigma_{\max} = 10$.
- Keeping the r largest singular values gives the best rank- r approximation of $\mathbf{W}$. Word embeddings are a low-rank factorization of a co-occurrence (PMI) matrix (W1); LoRA writes a weight update as $\mathbf{B}\mathbf{A}$ with a small inner dimension (W8).

Softmax. $\text{softmax}(\mathbf{z})_i = e^{z_i} / \sum_j e^{z_j}$ turns any real vector (“logits”) into a distribution. Adding a constant to every z_i changes nothing; dividing by a temperature τ sharpens ($\tau < 1$) or flattens ($\tau > 1$) it. Every language-model head ends in one (W1 onward), so do attention weights (W3), and “temperature” at inference time means exactly this τ (W10).

2. Probability — *UDL Appendix C; ch 5 (loss functions)*

The chain rule of probability is exact and assumption-free: $P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_{<t})$. A language model is nothing but a model of each factor. The **Markov assumption** — conditional independence of w_t from everything before the last $n - 1$ tokens — is what an n-gram model *adds*, and it is what W1 shows breaking. Transformers drop the assumption but keep the factorization.

Expectation and variance. $\mathbb{E}$ is linear always; $\text{Var}(\sum_i X_i) = \sum_i \text{Var}(X_i)$ needs independence. A dot product of two d -dimensional vectors with independent unit-variance entries has variance d — which is why attention divides by $\sqrt{d_k}$ (W3), why initialization scales with width (W4), and why a benchmark score on n items has standard error $\sqrt{p(1-p)/n}$ (W14: on 500 items at $p = 0.7$ that is about 2 points, so a one-point gap is not a result).

Maximum likelihood = minimizing negative log-likelihood.

$$\hat{\theta} = \arg \max_{\theta} \prod_t P_{\theta}(w_t \mid w_{<t}) = \arg \min_{\theta} \sum_t -\log P_{\theta}(w_t \mid w_{<t}).$$

The log turns a product into a sum, and a sum of per-token terms is what you can differentiate and average over a batch. For an n-gram model the maximizer is just *counts divided by counts* (W1). For a Transformer it is the same objective, optimized by gradient descent — the objective never changes from W1 to W7.

Entropy, cross-entropy, KL. For distributions p (data) and q (model):

$$H(p) = -\mathbb{E}_p[\log p], \quad H(p, q) = -\mathbb{E}_p[\log q] = H(p) + D_{\text{KL}}(p \parallel q), \quad D_{\text{KL}}(p \parallel q) \geq 0.$$

The per-token training loss *is* $H(p, q)$ estimated on data; training can only shrink the KL term, so the loss has a floor $H(p)$ it will never reach (W1, W7). With $\log_2$, $H(p, q)$ is the average number of bits needed to encode the data using q — the “compression = prediction” argument of W1 — and perplexity is $2^{H(p, q)}$. KL returns as the penalty that keeps a fine-tuned policy near its reference model (W9) and inside the contrastive loss of W12.

Sigmoid and pairwise preference. $\sigma(x) = 1/(1 + e^{-x})$, with $1 - \sigma(x) = \sigma(-x)$. “A beats B with probability $\sigma(r_A - r_B)$ ” (Bradley–Terry) is the whole probabilistic content of reward models and DPO (W9); the same σ is the classifier inside word2vec’s negative sampling (W1).

3. Calculus and optimization — UDL ch 6 (*fitting models*), ch 7 (*gradients*)

Gradient descent. $\nabla_\theta \mathcal{L}$ points in the direction of steepest increase, so $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$ decreases $\mathcal{L}$ for small enough η . *Stochastic* gradient descent estimates the gradient on a mini-batch: unbiased, but noisy, and the noise level is set by batch size (W4, W7). Adam rescales each coordinate by a running estimate of its variance — which is why it stores two extra numbers per weight, the $2N$ optimizer states in the ledger (W1, W4).

Chain rule and Jacobians. For vector-valued functions $\mathbf{y} = f(\mathbf{x})$ the derivative is a matrix, the Jacobian $\partial \mathbf{y} / \partial \mathbf{x}$, and the derivative of a composition is the *product of the Jacobians*. Backpropagation is the chain rule organized so that each Jacobian is used once, from the loss backward; it costs about twice the forward pass, which is where the 6 in $6ND$ comes from (W1). UDL ch 7 works this out on a toy network; do it once by hand.

Products of Jacobians through depth. A network of depth T (or a recurrent net unrolled T steps) has a gradient that is a product of T Jacobians. By §1, its norm is bounded by $\prod_t \sigma_{\max}$: with $\gamma \sigma_{\max} = 0.9$ per step, 50 steps leave $0.9^{49} \approx 0.006$; with 1.1, they give ≈ 107 . Nothing about $\mathbf{W}$ needs to be extreme; a sentence is long enough (W1). The cure is an *additive* path whose Jacobian is $\mathbf{I} +$ (something small) — the LSTM cell state along time, the residual connection along depth (W1 $\rightarrow$ W4). This single idea explains why every modern architecture is a residual stream.

Three derivatives you will see again and again.

$$\frac{d}{dx} \log \sigma(x) = \sigma(-x), \quad \nabla_{\mathbf{z}} [-\log \text{softmax}(\mathbf{z})_y] = \text{softmax}(\mathbf{z}) - \mathbf{e}_y, \quad \frac{d}{dx} \log x = \frac{1}{x}.$$

The first drives the word2vec derivation (W1) and the DPO gradient (W9). The second says the gradient of a language model’s loss at the logits is simply *predicted minus actual* — every token’s training signal is that vector.

Self-check

If you can answer these without looking anything up, you do not need the reading. Each takes one or two lines.

1. Two random vectors in $\mathbb{R}^d$ have i.i.d. entries of mean 0 and variance 1. What is the variance of their dot product, and what should you divide by to make it 1?
2. Show that $H(p, q) \geq H(p)$, and name the quantity that is the difference.
3. Give a 2×2 matrix whose eigenvalues are both 0 but which stretches some vector by a factor of 10. Why does this matter for a product of 50 different matrices?
4. A recurrent net has $\sigma_{\max}(\mathbf{W}) = 0.8$ and uses tanh (so $\gamma = 1$). Bound the gradient norm through 30 time steps. Is the bound an upper bound, a lower bound, or both?
5. Differentiate $\log \sigma(x)$ and $\log \sigma(-x)$ with respect to x .
6. A model assigns per-token probabilities $q_1, \dots, q_T$ to a sequence of T tokens occupying B bytes. Write the total code length in bits, then the bits per byte. Which of the two can be compared across two tokenizers, and why?

Reading order, if you need all of it: UDL Appendix B $\rightarrow$ C $\rightarrow$ ch 5 $\rightarrow$ 6 $\rightarrow$ 7, about a chapter a day; then UDL ch 12 or ZZM ch 4 for a first look at attention before Week 3.